

SOFTWARE TESTING SEMINAR

API Testing & Contract Testing

Nhóm 03 — SEBros

THÀNH VIÊN

Ngô Nguyễn Thế Khoa
23127065

Mạch Quốc Tấn
23127115

Ân Tiến Nguyễn An
23127148

Nguyễn Tuấn Anh
23127152

Nguyễn Lê Hồ Anh Khoa
23127211

Postman

Newman

Pact

GitHub Actions

Nội dung trình bày

- 01 Giới thiệu & Mục tiêu
- 03 Công cụ: Postman & REST Client
- 05 Automation với Newman & CI/CD
- 07 Pact: Consumer → Provider → Broker
- 09 AI hỗ trợ kiểm thử
- 02 API Testing — Khái niệm & Kỹ thuật
- 04 Demo: API Testing thực hành
- 06 Contract Testing — Vấn đề & Giải pháp
- 08 Demo: Contract Testing & Breaking Change
- 10 Tổng kết & Q&A

Giới thiệu & Mục tiêu

Bối cảnh

- Hệ thống hiện đại chia thành nhiều **service độc lập** (microservices)¹
- Mỗi service có API riêng → cần kiểm thử ở **nhiều lớp**
- Kiểm thử thủ công không đủ → cần **automation & CI/CD**²

Mục tiêu seminar

- Hiểu và thực hành **API Testing** (functional testing tầng API)
- Hiểu và thực hành **Contract Testing** (tương thích Consumer–Provider)
- Tự động hóa bằng **Newman + GitHub Actions**
- Trải nghiệm quy trình **AI-assisted testing**

Phạm vi: Product Service API (Node.js/Express) — CRUD 5 endpoints · Postman, Newman, Pact, GitHub Actions

¹ **microservices** — Kiến trúc chia hệ thống thành các service nhỏ, chạy độc lập, giao tiếp qua API.

² **CI/CD** — Tự động hóa liên tục: tích hợp mã (CI) và triển khai (CD) mỗi khi có thay đổi.

PHẦN A — API TESTING

Khái niệm · Kỹ thuật · Công cụ · Demo

API là gì?

API (Application Programming Interface): giao diện lập trình cho phép các hệ thống giao tiếp.

REST API: dùng HTTP protocol — method, URL, headers, body.

Request: Method + URL + Headers + Body

Response: Status Code + Headers + Body

HTTP Methods

Method	Ý nghĩa	Ví dụ
GET	Đọc dữ liệu	GET /products
POST	Tạo mới	POST /products
PUT	Cập nhật toàn bộ	PUT /product/10
DELETE	Xóa	DELETE /product/10

HTTP Status Codes

Mã	Ý nghĩa
200	OK — thành công
201	Created — tạo mới thành công
204	No Content — xóa thành công
400	Bad Request — dữ liệu không hợp lệ
401	Unauthorized — thiếu/sai token
404	Not Found — không tồn tại

API có Authentication vs Không

Không authenticate

- Truy cập tự do, không cần token
- Ví dụ: public API thời tiết, health check endpoint

Có authenticate (Token-based)

- Yêu cầu header: `Authorization: Bearer <token>`
- Token có thể là JWT, OAuth2, hoặc custom (ISO-8601)¹
- Thiếu/sai/hết hạn token → `401 Unauthorized`

Ví dụ trong Product Service:

```
Authorization: Bearer 2026-07-15T10:00:00.000Z
```

Timestamp phải trong vòng **1 giờ** so với server · Sai định dạng hoặc hết hạn → `401`

¹ **Bearer ISO-8601** — Token dạng timestamp theo chuẩn ISO-8601 (vd: 2026-07-15T10:00:00.000Z), hợp lệ trong 1 giờ.

Các loại Test Case cho API

Loại	Mô tả	Ví dụ
Happy Path ¹	Request hợp lệ → response đúng	GET /product/10 + valid token → 200
Negative	Input sai → xử lý lỗi đúng	GET /product/99999 → 404
Authentication	Thiếu/sai/hết hạn token → 401	Không gửi Authorization header
Validation	Body thiếu field → 400	POST thiếu "name" → 400
Schema	Response đúng cấu trúc kỳ vọng	Body có đủ id, type, name, version
Boundary	Giá trị biên	Token đúng 1 giờ trước (vừa hết hạn)

Kỹ thuật: Domain Partitioning · Boundary Value Analysis · State Transition

¹ Happy Path — Kịch bản với dữ liệu hợp lệ, đi qua luồng thành công.

Postman — Tổng quan

Postman là gì

- Nền tảng kiểm thử API **phổ biến nhất**
- Hỗ trợ Desktop App, Web, và **VS Code Extension**
- Miễn phí cho cá nhân & nhóm nhỏ (≤ 3 người)
- **Charge phí doanh nghiệp khi:**
 - Team size > 3 thành viên
 - Vượt quota Cloud (Mock, Monitor...)

Các khái niệm chính

- **Collection** — Gom nhóm request
- **Environment** — Biến môi trường
- **Variable** — Giá trị động dùng lại
- **Pre-request Script** — Chạy trước request
- **Test Script** — Assertion¹ chạy sau response

Tính năng nâng cao

- **Collection Runner** — Chạy chuỗi request
- **Data-driven** — Chạy test với file (CSV/JSON)
- **Newman** — CLI runner cho CI/CD
- **Monitor** — Test định kỳ tự động
- **Mock Server** — Giả lập API nhanh

¹ **Assertion** — Câu lệnh kiểm tra tự động (vd: status = 200) chạy sau mỗi request.

Postman — Tổ chức Collection

Product Service – Data Driven Tests

- `_Setup (Pre-flight)` ← sinh token
- `GET – Happy Path` ← 4 iterations
- `GET – Negative` ← 7 iterations
- `POST – Happy Path` ← 2 iterations
- `POST – Negative` ← 5 iterations
- `PUT – Happy Path` ← 2 iterations
- `PUT – Negative` ← 4 iterations
- `DELETE – Happy Path` ← 1 iteration
- `DELETE – Negative` ← 4 iterations

29

Test cases

5

Endpoints

9

Folders

Postman — Script & Assertion

Pre-request Script (Collection level)

Tự động sinh Bearer token hợp lệ mỗi iteration:

- `"{{validToken}}"` → Bearer hợp lệ
- `"Bearer 2020-..."` → token hết hạn (negative)
- `""` → xóa header (no token case)

Test Script (ví dụ)

```
pm.test("Status is 200", () => {  
    pm.response.to.have.status(200);  
});  
  
pm.test("Response has required fields", () => {  
    const json = pm.response.json();  
    pm.expect(json).to.have.property("id");  
    pm.expect(json).to.have.property("name");  
    pm.expect(json).to.have.property("type");  
});
```

Data-driven Testing

Khái niệm: Chạy cùng 1 request với **nhiều bộ dữ liệu** khác nhau. Data file: JSON hoặc CSV. Mỗi dòng = 1 iteration.

```
{
  "tc_id": "GET_ID_01",
  "description": "Existing product with valid token",
  "product_id": "10",
  "auth_header": "{{validToken}}",
  "expected_status": 200,
  "expect_field_id": "10",
  "expect_field_name": "28 Degrees"
}
```

Tách logic khỏi data

Script không đổi, chỉ thêm data

Dễ thêm case mới

Thêm dòng vào data file

Phù hợp automation

Chạy với Newman CI/CD

VS Code REST Client

REST Client (extension)

- File `.http` — viết request ngay trong VS Code
- Không cần mở Postman GUI
- Hỗ trợ biến, chaining, và assertion cơ bản

```
@baseUrl = http://localhost:8080
@token = {{$datetime iso8601 -1 m}}

### GET /products → 200
GET {{$baseUrl}}/products
Authorization: Bearer {{$token}}
```

So sánh nhanh

Tiêu chí	Postman	REST Client
GUI	Đầy đủ (App/Web/Extension)	Tối giản
Data-driven	✓	✗
Script	JS đầy đủ	Hạn chế
CI/CD	Export → Newman	✗
Tiện lợi	Có App + Extension	Ngay trong editor

API mẫu: Product Service

Thông tin

- **Node.js + Express**
- Port: 8080
- Auth: **Bearer ISO-8601** (trong vòng 1 giờ)
- Data: **In-memory** (reset khi restart)

Product Schema

```
{
  "id": "10",
  "type": "CREDIT_CARD",
  "name": "28 Degrees",
  "version": "v1"
}
```

Endpoints

Method	Path	Success	Error
GET	/products	200 + array	401
GET	/product/:id	200 + object	401, 404
POST	/products	201 + created	400, 401
PUT	/product/:id	200 + updated	401, 404
DELETE	/product/:id	204	401, 404

PHẦN B — AUTOMATION & CI/CD

Newman · GitHub Actions · Pipeline

Newman — CLI Runner

Newman là gì

- Command-line runner¹ cho **Postman Collection**
- Chạy test **không cần mở Postman GUI**
- Xuất báo cáo: **CLI, HTML, JSON**

Luồng tự động hóa

1. Khởi động Provider tại `localhost:8080`
2. Readiness probe²: gọi `/health`
3. Chạy Newman với collection + environment
4. Xuất báo cáo HTML + JSON
5. Upload artifact³ (kể cả khi test fail)

Lệnh cốt lõi

```
newman run collection.json \  
-e environment.json \  
--reporters cli,htmlextra,json \  
--reporter-htmlextra-export report.html \  
--reporter-json-export report.json
```

¹ **CLI runner** — Công cụ chạy test trực tiếp từ dòng lệnh, không cần giao diện.

² **readiness probe** — Request kiểm tra service đã khởi động sẵn sàng (vd: /health).

³ **artifact** — File đầu ra (report, log) được CI lưu lại để xem sau.

GitHub Actions — Newman Pipeline

Workflow: `newman-api-test.yml` | Trigger: `push/PR vào main + manual dispatch`

Checkout → Setup Node 20 → Install deps → Start Provider
→ Wait `/health` (30s timeout) → Install Newman
→ `Run Newman` → Upload report artifact

permissions: contents: read

Giới hạn quyền tối thiểu

timeout-minutes: 10

Tránh pipeline treo

concurrency: cancel-in-progress

Hủy run cũ khi push mới

if: always()

Luôn upload report để debug

GitHub Actions — Pact Pipeline

Workflow: `pact-verification.yml`

Consumer Pact

Sinh pact.json
npm run test:pact



Provider Verification

Verify pact thật
Chạy với API thật



can-i-deploy¹

Quality gate²
Chặn nếu incompatible

Job 1 — Consumer

Chạy consumer test → sinh pact JSON →
upload artifact → publish lên Broker

Job 2 — Provider

Download pact → chạy verifier với API thật →
publish kết quả

Job 3 — can-i-deploy

Kiểm tra compatibility matrix → chặn deploy
nếu chưa tương thích

¹ **can-i-deploy** — Lệnh của Pact Broker: kiểm tra phiên bản này có an toàn để deploy.

² **quality gate** — Cổng kiểm tra tự động — chặn pipeline nếu không đạt.

Giá trị của Automation

Hai lớp bảo vệ

Lớp	Công cụ	Phát hiện
Functional	Newman/Postman	Lỗi chức năng, validation, auth, payload
Compatibility	Pact	Breaking change tại biên Consumer–Provider

Phản hồi sớm

Chạy trên mỗi push/PR

Tái lập

Log + artifact lưu lại để debug

Giảm thủ công

Không cần kiểm tra manual

Bảng chứng

Reviewer truy ngược version, test result

PHẦN C — CONTRACT TESTING

Vấn đề · Giải pháp · Pact Framework

Khi mỗi service đều "xanh"... hệ thống vẫn có thể "đỏ"

Consumer¹

Kỳ vọng field `product.name`, nhưng
Provider đổi thành `displayName`

Provider²

Unit test vẫn pass — logic và schema
mới đều đúng theo góc nhìn backend

Runtime

Lỗi chỉ lộ khi tích hợp — trên staging
hoặc production

"Hai phía có còn hiểu cùng một giao thức hay không?"

Unit test kiểm tra logic nội bộ — không kiểm chứng giả định xuyên biên giới

¹ **Consumer** — Bên gọi API (vd: FrontendWebsite).

² **Provider** — Bên cung cấp API (vd: ProductService).

Contract Testing là gì?

*Contract¹ là đặc tả **có thể thực thi** về những request Consumer gửi và response Provider cam kết đáp ứng.*

- **Request:** method, path, query, headers, body
- **Response:** status, headers, schema và matching rules
- **Context:** provider state² — điều kiện trước của interaction³

INTERACTION

Given product 10 exists
When GET /product/10
Then 200 + Product schema

Contract Testing xác minh **tính tương thích** — không chứng minh toàn bộ nghiệp vụ đúng.

¹ **Contract** — Bản cam kết bằng máy về cách hai bên giao tiếp.

² **Provider state** — Điều kiện dữ liệu cần có trước khi chạy interaction (vd: "product 10 exists").

³ **Interaction** — Một cặp request–response cụ thể trong contract.

So sánh các lớp kiểm thử

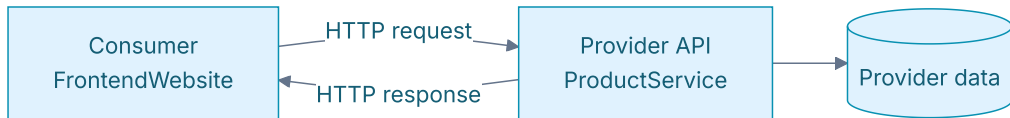
Tiêu chí	API Testing	Contract Testing	Integration ¹	E2E ²
Câu hỏi	Endpoint đúng?	Còn tương thích?	Phối hợp đúng?	Journey chạy?
Phạm vi	1+ endpoint	1 cặp C-P	Nhóm thành phần	Toàn hệ thống
Môi trường	API thật/mock	Cô lập, local/CI	Bán tích hợp	Gần production
Feedback	Nhanh-TB	Nhanh, rõ	Trung bình	Chậm
Điểm mạnh	Chức năng, auth	Chống breaking change	Wiring	User journey
Điểm mù	Phụ thuộc consumer	Business logic	Ngoài scope	Flaky, đắt

Contract Test **bổ sung** Unit / Integration / E2E — không thay thế tất cả.

¹ **Integration** — Kiểm thử nhiều thành phần phối hợp với nhau.

² **E2E** — Kiểm thử toàn bộ hành trình người dùng qua nhiều service.

Mô hình Consumer-Provider



Consumer (FrontendWebsite)

Mô tả chính xác phần API nó sử dụng → sinh Pact file (JSON)

Provider (ProductService)

Chứng minh implementation đáp ứng mọi interaction → chạy verification

Pact Broker:¹ Lưu contract + version + kết quả verification → Compatibility matrix² → `can-i-deploy` gate

¹ **Pact Broker** — Kho trung tâm lưu contract + kết quả verification.

² **Compatibility matrix** — Bảng tra cứu phiên bản Consumer nào tương thích phiên bản Provider nào.

Consumer-Driven: Vì sao?

Provider-Driven

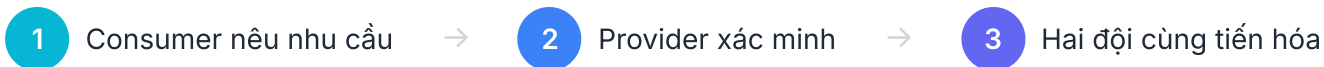
Provider công bố toàn bộ schema. Consumer phải thích nghi.

Vấn đề: Provider khó biết phần nào đang được sử dụng.

Consumer-Driven

Mỗi Consumer phát hành interaction mình phụ thuộc.
Provider xác minh tập hợp nhu cầu thực tế.

Lợi ích: Provider biết field nào đang dùng → tránh xóa/sửa nhầm.



Giới hạn của Contract Testing

Business logic nội bộ

Response đúng schema nhưng tính sai tổng tiền vẫn pass

Hạ tầng production

DNS, TLS, gateway, timeout cần lớp test/monitor khác

Hành trình nhiều service

Một pact chỉ chứng minh một ranh giới

Chất lượng API design

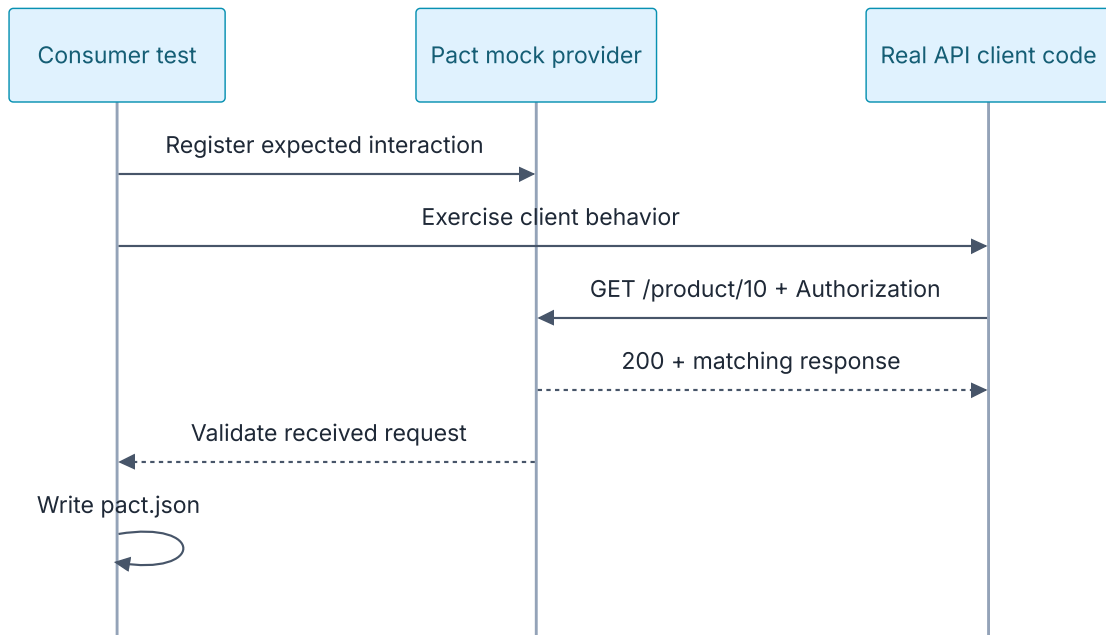
Tương thích ≠ dễ dùng, nhất quán, an toàn

Unit → logic · **Contract** → compatibility · **Integration** → wiring · **E2E** → critical journeys

PHẦN D — DEMO PACT

Consumer → Provider → Broker → Breaking Change

Bước 1 — Consumer tạo contract



Mock Provider¹ không thay thế code Consumer — test phải gọi **API client thật** của Consumer.

¹ **Mock provider** — Server giả do Pact dựng lên để Consumer test chạy mà không cần Provider thật.

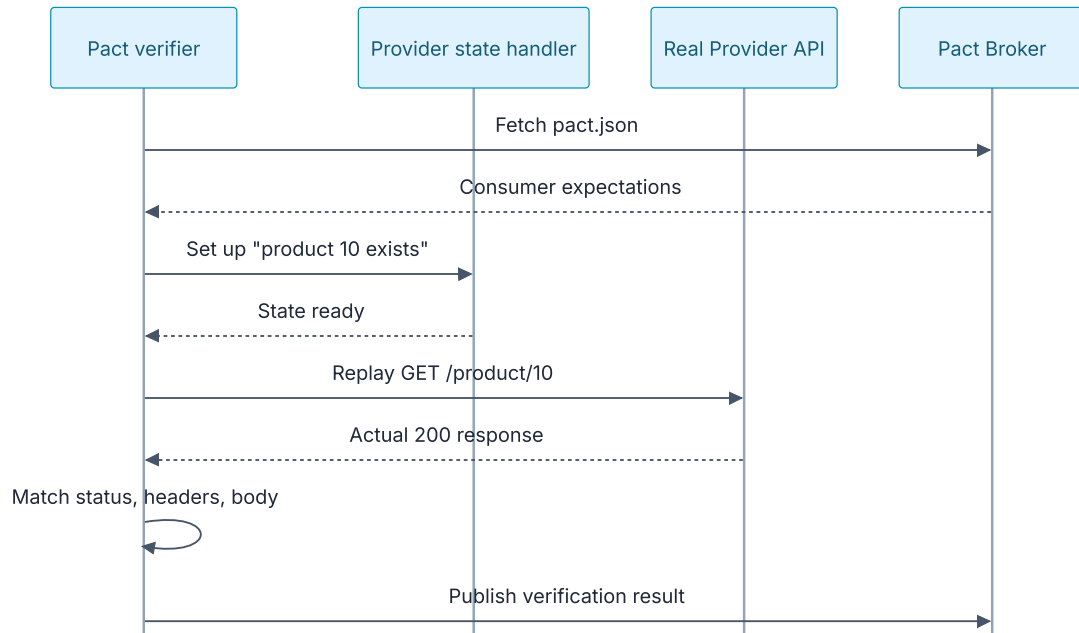
Pact JSON — Cấu trúc Interaction

```
{
  "description": "get product 10",
  "providerState": "product 10 exists",
  "request": {
    "method": "GET",
    "path": "/product/10"
  },
  "response": {
    "status": 200,
    "body": {
      "id": "10",
      "name": "28 Degrees",
      "type": "CREDIT_CARD"
    }
  },
  "matchingRules": {
    "$.body.id": "type:string",
    "$.body.name": "type:string"
  }
}
```

Nguyên tắc Matcher: Strict với điều Consumer phụ thuộc · Linh hoạt với dữ liệu động · Không over/under-specify

Repo: 10 interactions · 10 Authorization regex matchers (Bearer ISO-8601)

Bước 2 — Provider xác minh contract¹



Real routing

Request đi vào API thật

Controlled state

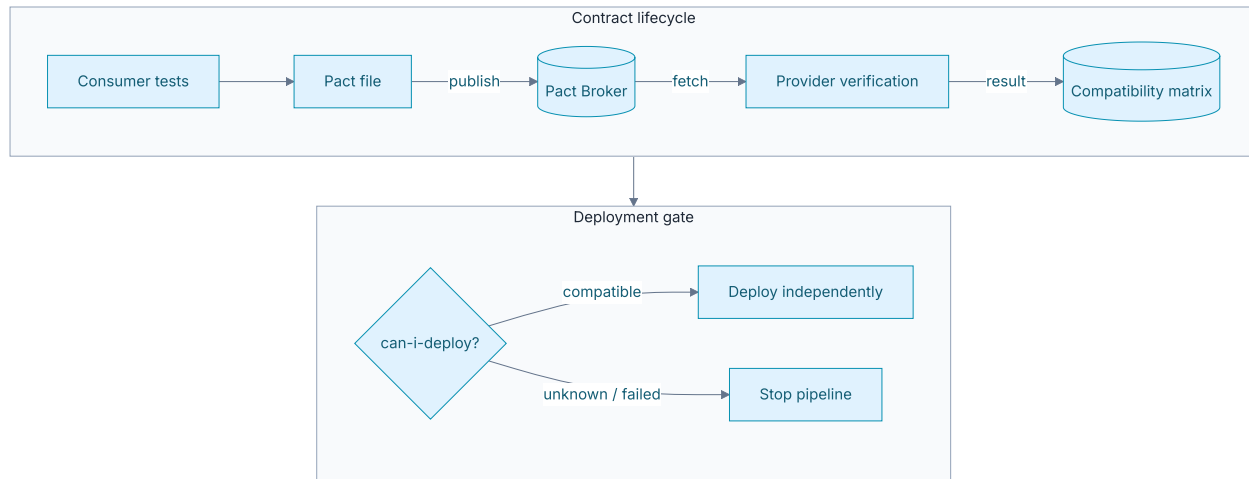
Dữ liệu test có thể tái tạo

Precise diff

Mismatch chỉ rõ field bị gãy

¹ **Verifier** — Công cụ Pact chạy phía Provider, replay từng request trong contract.

Broker & Deployment Gate¹



Broker liên kết **version Consumer + version Provider + kết quả verification** → Compatibility matrix

¹ **Deployment gate** — Điểm kiểm soát quyết định có được deploy hay không.

Case Study: Product Service

Bộ contract (10 interactions)

Nhóm API	Số	Trạng thái
GET /products	2	Có dữ liệu + danh sách rỗng
GET /product/:id	2	Tồn tại + không tồn tại
POST /products	2	Tạo thành công + validation error
PUT /product/:id	2	Cập nhật + không tồn tại
DELETE /product/:id	2	Xóa thành công + không tồn tại

Kết quả

10/10Consumer interactions
pass

Provider verification pass

Lệnh chạy

```
# Consumer test
npm run test:pact --prefix .../consumer

# Provider verification
npm run test:pact --prefix .../provider
```


Breaking Change Demo

Kịch bản

- Provider đổi field `name` → `title` trong response
- HTTP status vẫn **200** — API "có vẻ" hoạt động
- Nhưng Consumer contract yêu cầu field `name`

Kết quả

- Provider verification **FAIL** — thiếu field `name`
- Pact chỉ rõ mismatch: expected `name`, got `title`
- Khôi phục `name` → verification **PASS** trở lại

BÀI HỌC

Đổi tên field là **breaking change**¹ dù HTTP status không đổi.

Functional test có thể **không phát hiện** nếu không assert đúng field.

Contract test phát hiện **ngay tại provider verification**.

¹ **Breaking change** — Thay đổi làm Consumer cũ bị gãy (vd: đổi tên field).

PHẦN E — AI HỖ TRỢ KIỂM THỬ

AI-assisted Testing · Agent Skill

AI trong quy trình Testing

Vai trò AI

- Sinh test case từ **API specification**¹
- Gợi ý cấu trúc **Postman Collection**
- Review contract, phát hiện **thiếu sót**
- Tạo sơ đồ, tài liệu, **workflow mẫu**

Công cụ đã dùng

- **Claude, ChatGPT, Gemini** — research & drafting
- **Postman Postbot** — sinh test script
- **Agent Skill**² — tự động sinh test từ API spec

¹ **API specification** — Tài liệu mô tả cấu trúc API (endpoint, tham số, response).

² **Agent Skill** — Tập hướng dẫn đóng gói để AI tự động sinh test.

Nguyên tắc AI-First

1. Hướng dẫn AI từng bước — không prompt chung chung

2. Human review mọi output — VALID / INVALID / INCOMPLETE

3. AI Audit Report — ghi log toàn bộ quá trình

4. Quality over completion

Agent Skill — AI-driven Test Generator

INPUT

API Specification
(OpenAPI¹ / Markdown)

PROCESS

Phân tích endpoint →
Sinh test cases →
Tạo Postman Collection

OUTPUT

Collection JSON +
Data files +
Test scripts

Tính tái sử dụng

>80%

Mã nguồn/prompt tái sử dụng

100%

Agent Skill, workflows, Newman runner

Demo: Chạy Agent Skill trên Swagger PetStore API → chứng minh reusability

¹ OpenAPI — Chuẩn mô tả API phổ biến (file YAML/JSON).

PHẦN F — TỔNG KẾT

So sánh · Khi nào dùng gì · Takeaway

API Testing vs Contract Testing

	API Testing	Contract Testing
Câu hỏi	Endpoint hoạt động đúng?	Consumer & Provider còn tương thích?
Công cụ	Postman + Newman	Pact
Phạm vi	Nhiều endpoint, nhiều case	Một cặp Consumer–Provider
Data	Data-driven (CSV/JSON)	Pact interactions + matchers
Automation	Newman trong GitHub Actions	Pact verification + can-i-deploy
Phát hiện	Lỗi chức năng, validation, auth	Breaking change tại biên
Bổ sung	✓ Cả hai cùng cần thiết	

Khi nào dùng gì?

Dùng API Testing khi

- Kiểm tra **chức năng** endpoint
- **Validation** input/output
- **Authentication** & Authorization
- **Regression suite**¹ cho API

Dùng Contract Testing khi

- Nhiều service phát triển **độc lập**
- Hay có **breaking change** khi tích hợp
- Cần **deployment gate** (can-i-deploy)
- Muốn **feedback sớm** trước staging

Chiến lược tối ưu: Unit → logic · Contract → compatibility · API/Integration → chức năng & wiring · Ít E2E → critical journeys

¹ **Regression suite** — Bộ test chạy lại mỗi khi có thay đổi để phát hiện lỗi cũ tái phát.

Adoption Path — Bắt đầu thế nào?

1 Chọn một ranh giới rủi ro
Cặp Consumer–Provider hay thay đổi/hay gây

2 Contract 1–2 luồng quan trọng
Success + một error behavior

3 Chạy trong CI
Consumer test + provider verification

4 Thêm Broker + can-i-deploy
Khi workflow ổn định

5 Đo hiệu quả
Lỗi phát hiện trước staging, thời gian feedback, số deploy bị chặn đúng

3 từ khóa

Fast

Feedback sớm tại local và CI

Focused

Khoanh đúng ranh giới Consumer–Provider

Safe

Các service tiến hóa độc lập có kiểm soát

API Testing + Contract Testing giải quyết hai nhóm rủi ro khác nhau.

Newman + GitHub Actions = regression suite tự động.

Pact = contract artifact + provider verification + compatibility gate.

Deliverables & Resources

Videos

- **Video 1:** Lý thuyết & Thuật ngữ API / Contract Testing
- **Video 2:** Hướng dẫn cài đặt môi trường
- **Video 3:** Demo thực hành (Postman + Newman + Pact & AI)

Tài liệu thực hành

- **Activity Worksheet** — 90 phút tại lớp
- **Mini Exercise** — Từ API Test đến Contract Breaking Change

Repository

- [src/sample-api/pact-workshop-js/](#) — Pact source code
- [src/postman/](#) — Collections + data files
- [src/newman/](#) — Runner scripts
- [.github/workflows/](#) — CI/CD pipelines

Tài liệu chính thức (Product Docs)

- [docs.pact.io](#) — Pact Foundation
- [learning.postman.com](#) — Postman
- [github.com/postmanlabs/newman](#) — Newman

Q&A & DISCUSSION

Hỏi & Đáp — Thảo luận

Seminar Kiểm thử Phần mềm — API Testing & Contract Testing

Mời Thầy/Cô & Các Bạn đặt câu hỏi 

THANK YOU

Chân Thành Cảm Ơn!

Cảm ơn Thầy/Cô và các bạn đã lắng nghe bài thuyết trình
của **Nhóm 03 — SEBros.**

Mạch Quốc Tấn

Ngô Nguyễn Thế Khoa

Nguyễn Lê Hồ Anh Khoa

Ân Tiến Nguyên An

Nguyễn Tuấn Anh